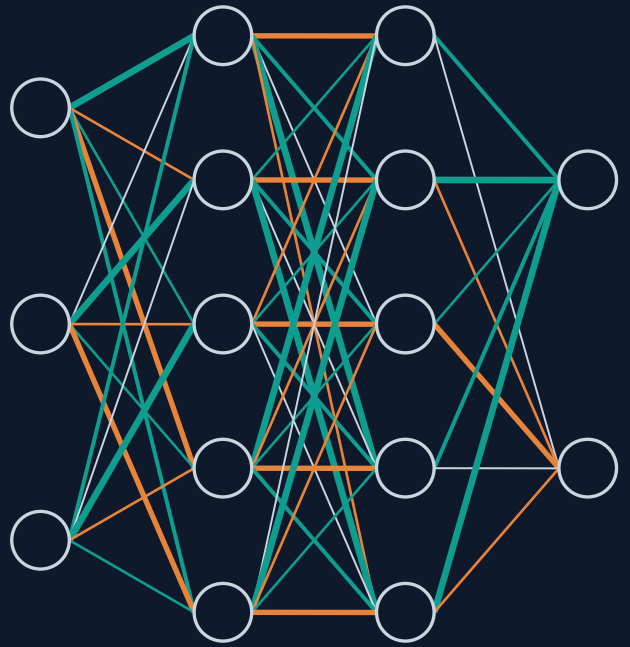
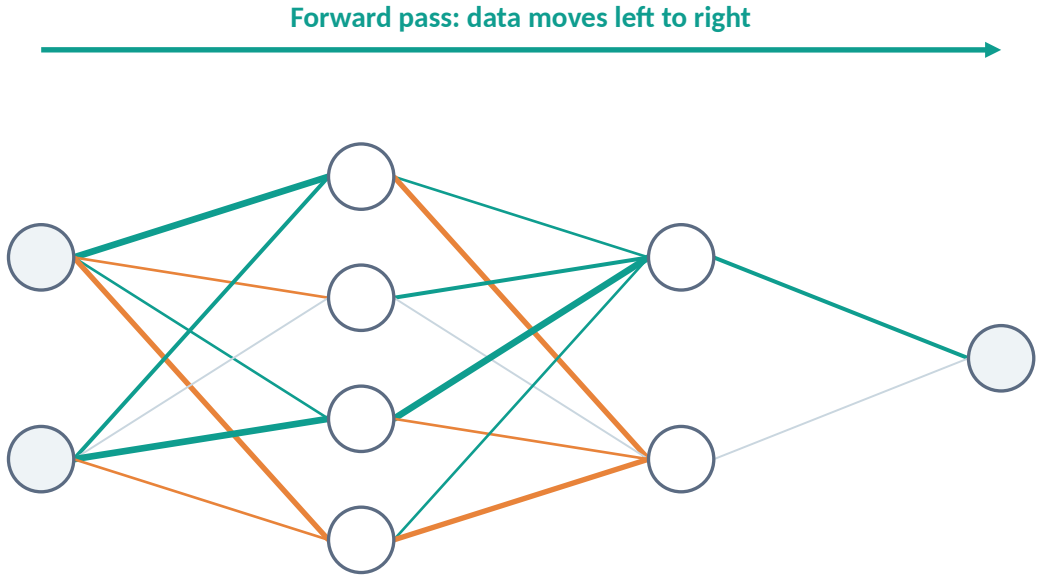
Inside a Neural Network

Six components, one learning loop

Teja Swaroop Cherukuri
AIML-501: Model Development, Indiana Wesleyan University
November 2026



How Data Flows Through a Network



Input layer
x_1, x_2

Hidden layer 1
4 neurons

Hidden layer 2
2 neurons

Output layer
prediction

1 Inputs enter
Each feature, such as an x and y coordinate, goes into one input neuron.

2 Hidden layers transform
Each neuron combines weighted inputs and applies an activation function.

3 Output predicts
The last layer gives the answer, which the loss function compares with the truth.

Source: Google for Developers (2025); network shape from Smilkov & Carter (n.d.).

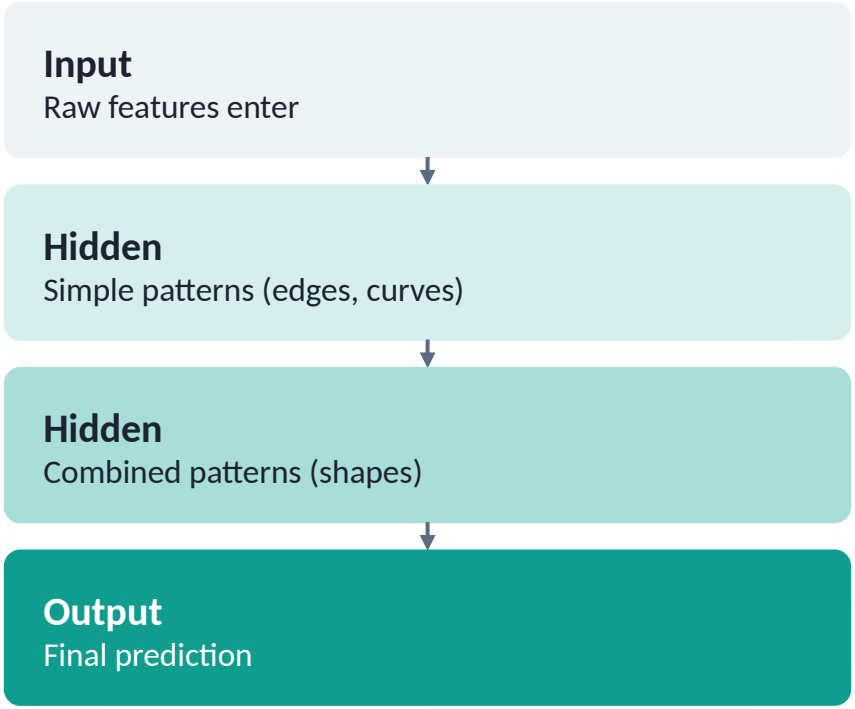
1. Layers

What they are

Groups of neurons that process data in stages. Every network has an input layer, one or more hidden layers, and an output layer.

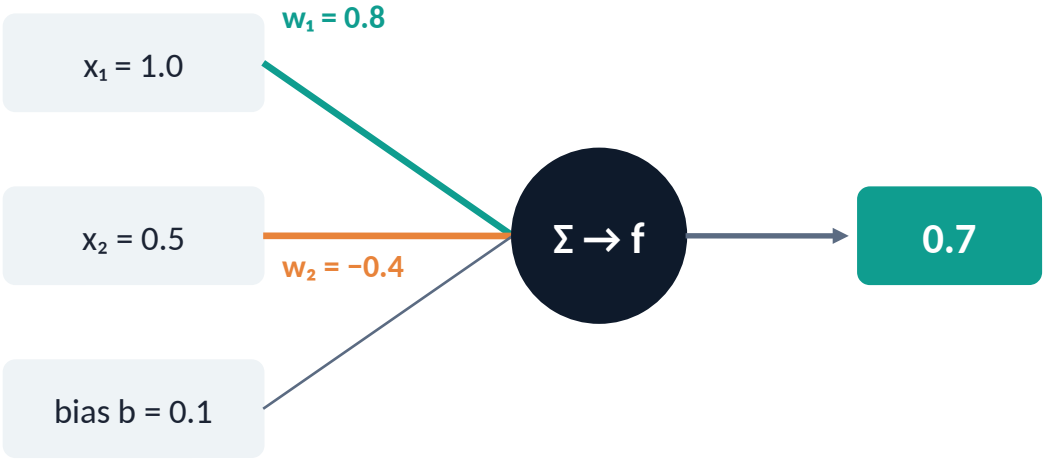
Why they matter

Each layer builds on the one before it. Early layers find simple patterns; deeper layers combine them into complex ones. More layers can learn harder shapes but take longer to train.



Source: Google for Developers (2025).

2. Neurons



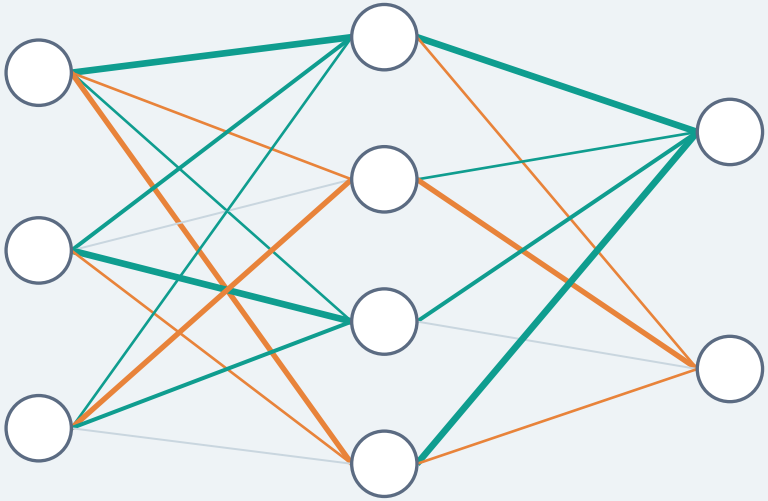
Worked example
z = (1.0)(0.8) + (0.5)(−0.4) + 0.1 = 0.7
output = ReLU(0.7) = 0.7

What it is
The basic unit of a network. It takes inputs, does one small calculation, and passes the result on.

How it works
1. Multiply each input by its weight
2. Add them up with a bias
3. Pass the total through an activation function

Source: Google for Developers (2025). Example values chosen for illustration.

3. Weights



Teal = positive weight **Orange** = negative weight Thicker = stronger

What they are

A number on every connection that controls how strongly one neuron's output affects the next neuron.

Positive vs. negative

A positive weight pushes the next neuron's value up; a negative weight pushes it down. A weight near zero means the connection barely matters.

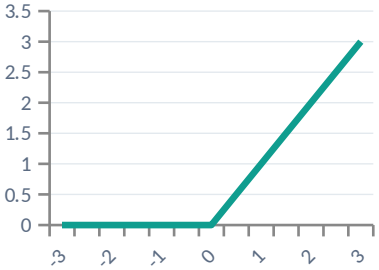
Why they matter

Weights are what the network learns. Training is the process of adjusting thousands or millions of weights until predictions are accurate.

Sources: Goodfellow et al. (2016); colors follow Smilkov & Carter (n.d.). Network weights are illustrative.

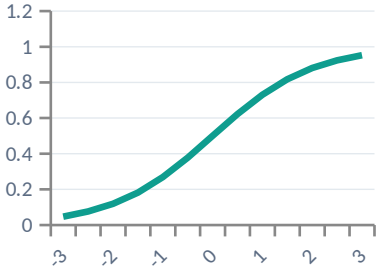
4. Activation Functions

ReLU



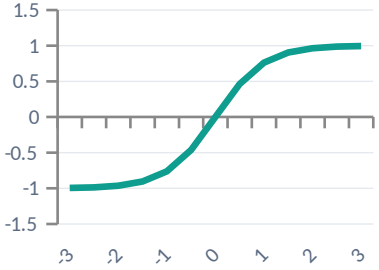
max(0, x). Fast and the most common choice for hidden layers.

Sigmoid



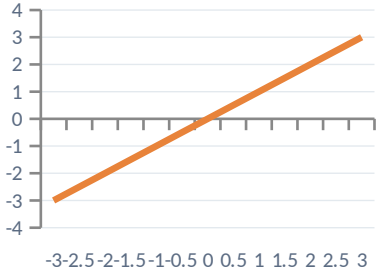
Squeezes any value into 0 to 1. Useful for probabilities.

Tanh



Squeezes values into −1 to 1. The Playground default.

Linear



Output equals input. With only linear activations, many layers act like one.

Curves calculated from each formula. Activations add the non-linearity that lets networks learn curved boundaries. Source: Google for Developers (2025).

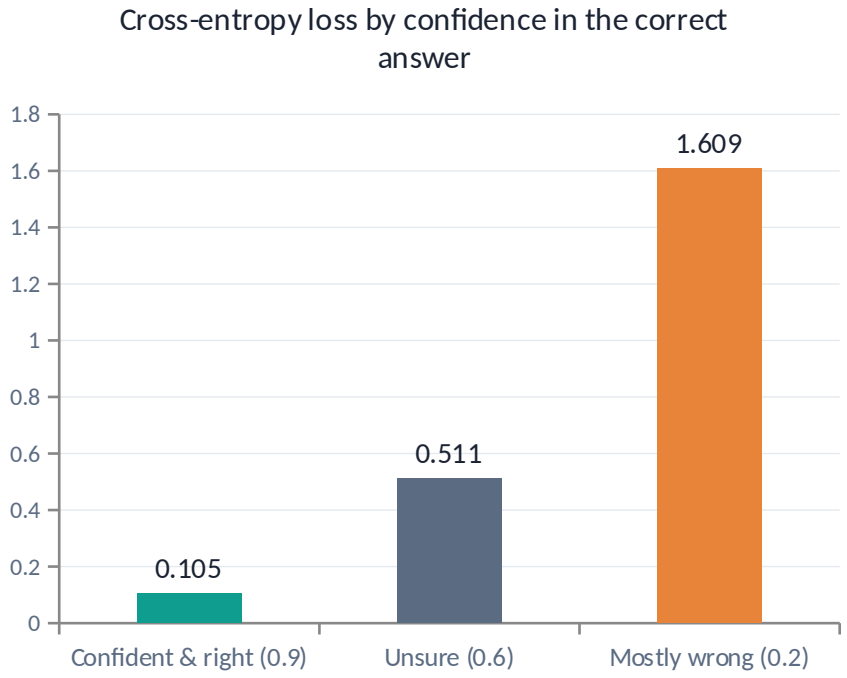
5. Loss Functions

What they are

A formula that scores how wrong a prediction is.
Training tries to make this number as small as possible.

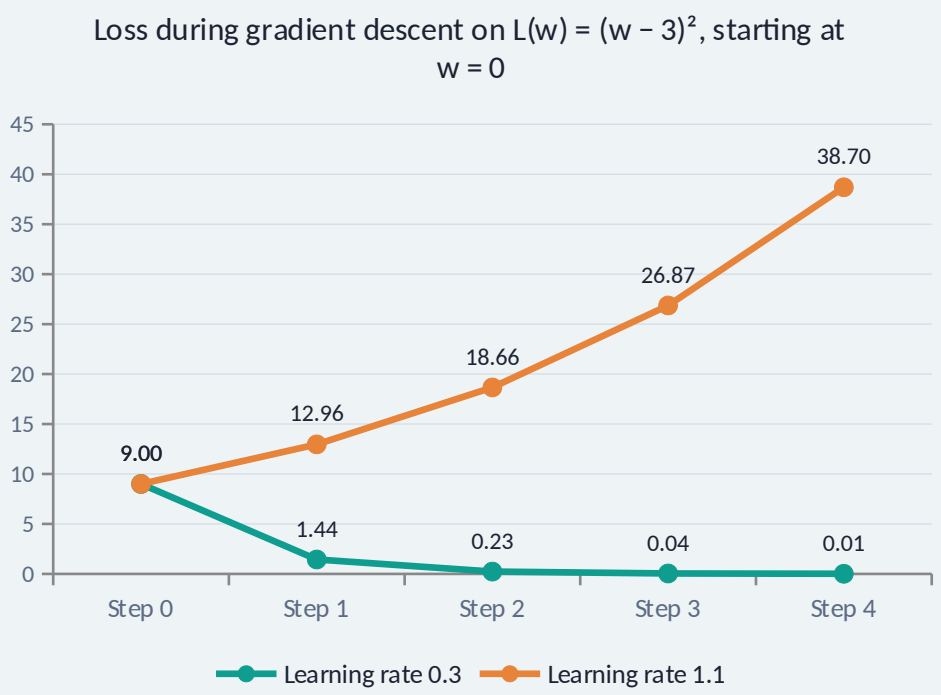
Two common types

Mean squared error (MSE): average of (prediction − truth)². Used for predicting numbers.
Cross-entropy: −ln(probability given to the correct answer). Used for classification.



Values calculated as −ln(p); loss grows fast when the model is confidently wrong. Source: Goodfellow et al. (2016).

6. Optimization Algorithms



What they are
Methods that adjust the weights to reduce the loss, one small step at a time.

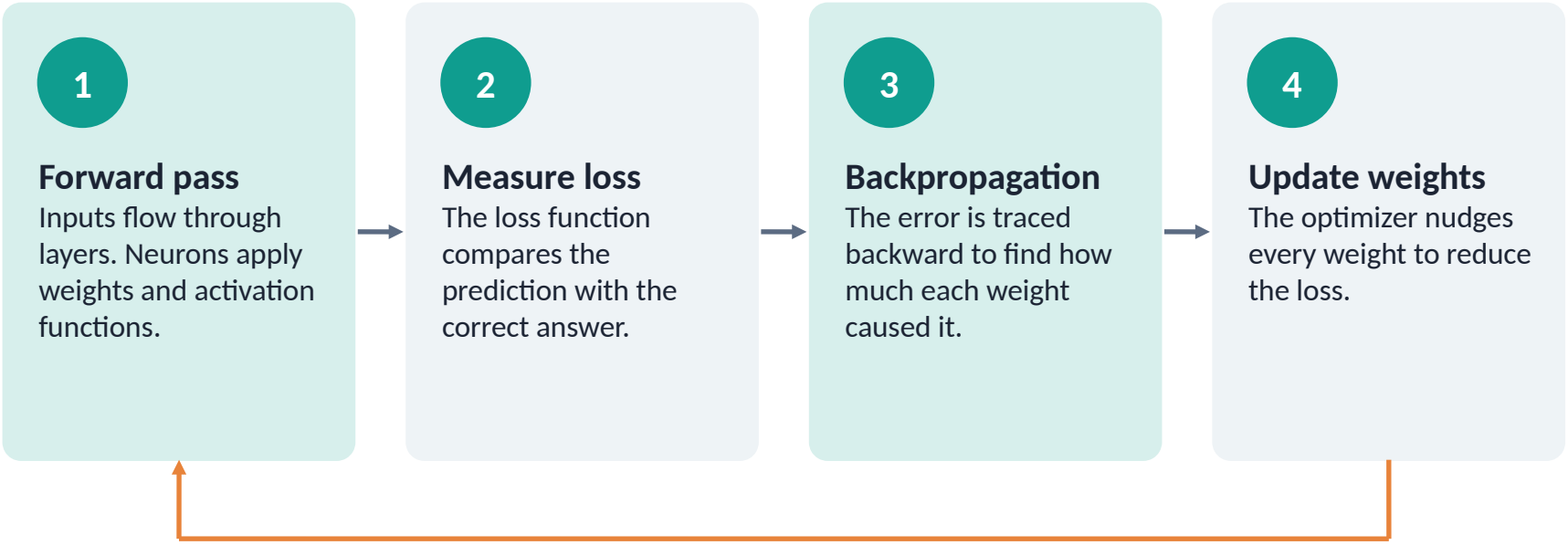
Gradient descent
Finds which direction lowers the loss and moves each weight a little that way. The step size is the learning rate.

Learning rate matters
At 0.3 the loss falls from 9.00 to 0.01 in four steps. At 1.1 each step overshoots and the loss grows to 38.70.

Common optimizers
SGD and Adam. Adam adapts the step size for each weight automatically.

Sources: Goodfellow et al. (2016); Kingma & Ba (2015). Chart values calculated, not measured.

The Learning Loop

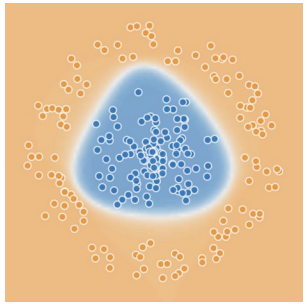


Source: Google for Developers (2025).

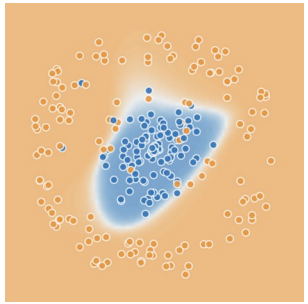
My Playground Experiments

Run	Dataset	Noise	Hidden layers	Activation	Epoch	Training loss	Test loss
1	Circle	0	4, 2	Tanh	311	0.003	0.004
2	Circle	30	4, 2	Tanh	311	0.089	0.182
3a	Spiral	0	4, 2	Tanh	524	0.409	0.452
3b	Spiral	0	6, 6, 6, 6	Tanh	512	0.275	0.295
4	Spiral	0	4, 2	Linear	327	0.470	0.474

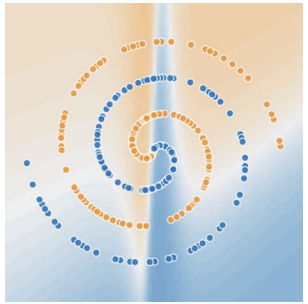
All runs: learning rate 0.03, batch size 10, 50% training data, no regularization, inputs X_1 and X_2, classification.



Run 1
Test loss 0.004



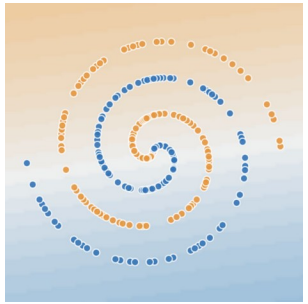
Run 2
Test loss 0.182



Run 3a
Test loss 0.452



Run 3b
Test loss 0.295



Run 4
Test loss 0.474

What My Experiments Showed



Noise widens the gap
Adding 30% noise raised test loss from 0.004 to 0.182, about twice its training loss (0.089). The network partly learned the noise.

Harder data needs more capacity
On the spiral, 4 layers of 6 neurons cut test loss from 0.452 to 0.295 at a similar epoch (512 vs. 524).

Activations create the curves
With Linear activation the network could only draw a straight boundary through the spiral. Test loss 0.474 was the worst of all runs.

Bigger is slower and bumpier
The 24-neuron network's loss curve became jagged late in training, so more capacity also needs more care.

Summary

Why visualizing networks matters

Seeing data flow, weights, and the decision boundary turns abstract math into something you can check with your own eyes.

Key insights

- All six components work as one learning loop.
- The right size network depends on the data.
- Non-linear activations are what make curved boundaries possible.
- Always compare test loss with training loss.

0.004

Test loss on the clean circle

0.452 → 0.295

Spiral test loss with a bigger network

0.474

Spiral test loss with Linear activation

References

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org/

Google for Developers. (2025, August 25). *Neural networks*. Machine Learning Crash Course. https://developers.google.com/machine-learning/crash-course/neural-networks

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980

Smilkov, D., & Carter, S. (n.d.). *A neural network playground* [Online tool]. TensorFlow. Retrieved November 7, 2026, from https://playground.tensorflow.org/